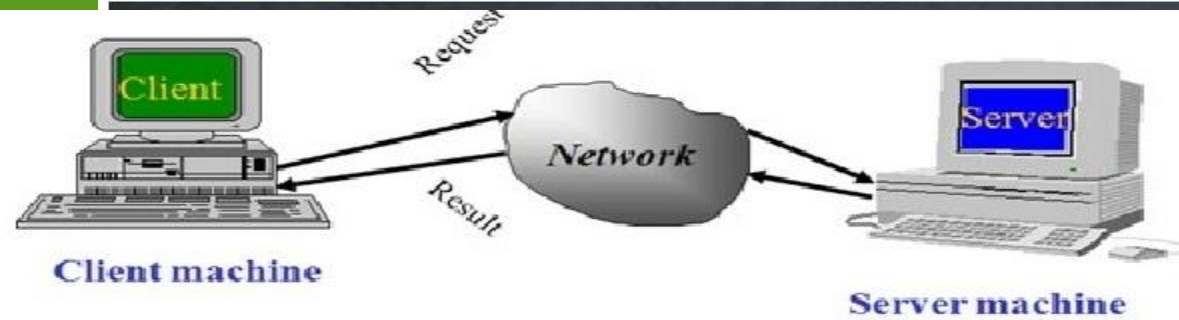


Network Programming in Java



Network Programming

Network Programming Topics

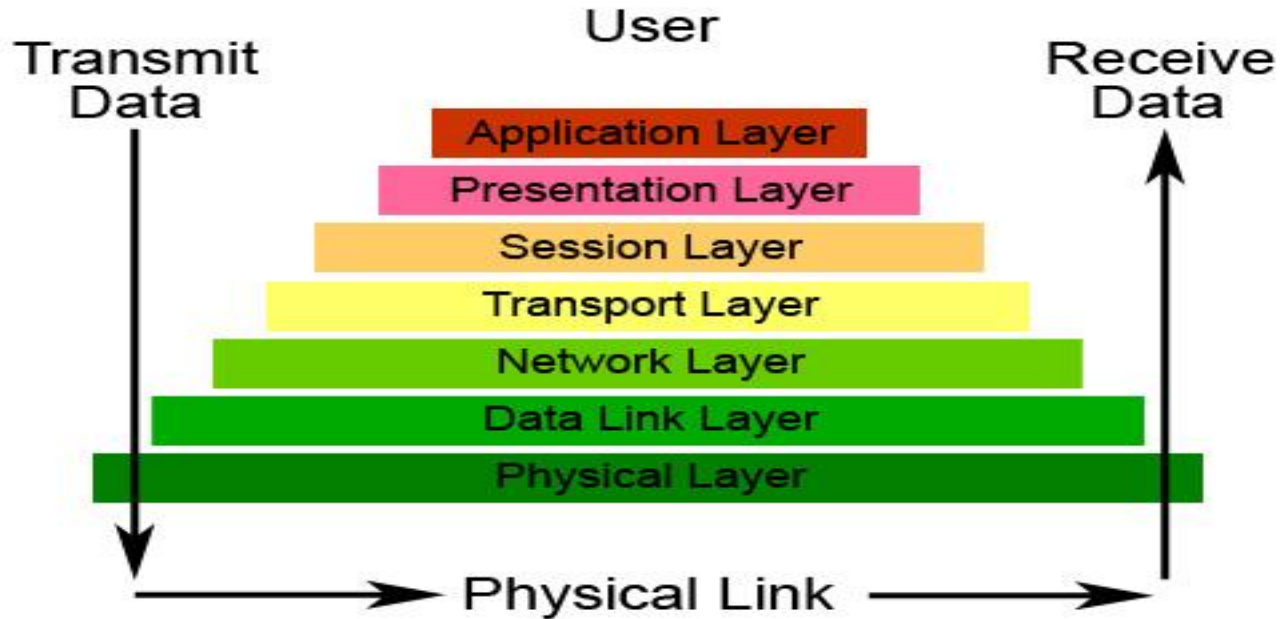
- ✓ Introduction and Overview
 - ✓ Network Programming Overview
 - ✓ TCP and UDP Overview
 - ✓ Socket Overview
- ✓ Network Programming in Java
 - ✓ Java.net Package Overview
 - ✓ InetAddress class
 - ✓ URL class
 - ✓ URLConnection
- ✓ TCP/IP based Socket programming
 - ✓ Server Socket class
 - ✓ Socket Class
- ✓ UDP based Socket Programming
 - ✓ Datagram Packet
 - ✓ Datagram Socket
- ✓ Assignments , Projects

Network Programming Overview

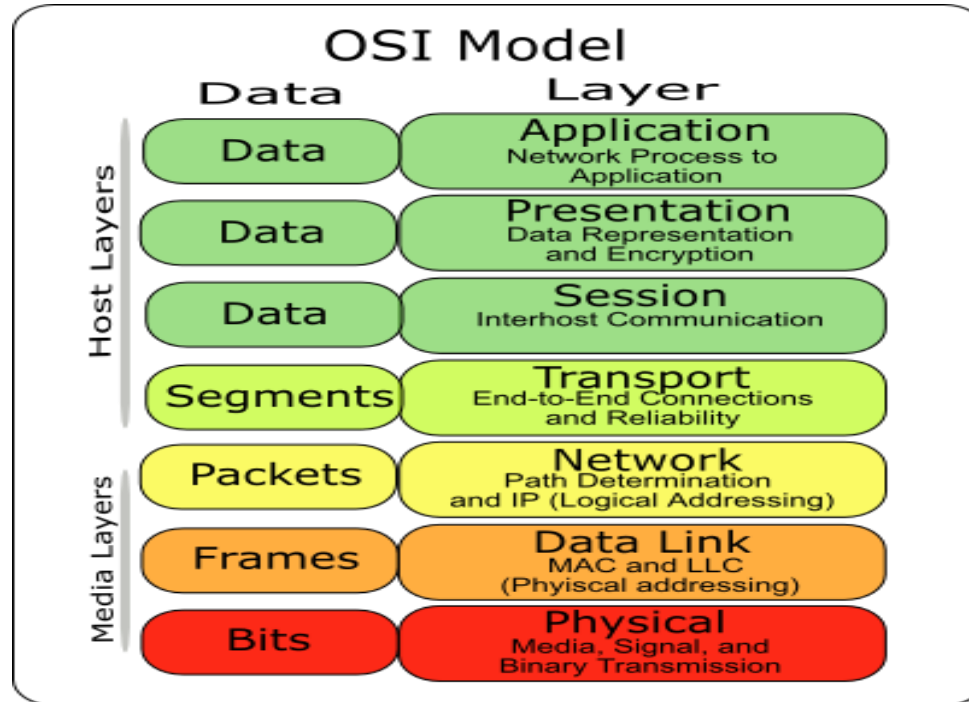
- ✓ What do you mean by Network and Network communication ?
- ✓ What is Client /Server Architecture ?
- ✓ What is Socket ?

OSI [Open System Interconnection] Model

The Seven Layers of OSI



OSI Model



TCP/IP and UDP Overview

- ✓ **TCP stands for Transmission Control Protocol, which allows for reliable communication between two applications.** TCP is typically used over the Internet Protocol, which is referred to as TCP/IP.
- ✓ **UDP stands for User Datagram Protocol, a connection-less protocol that allows for packets of data to be transmitted between applications.**

TCP/IP and UDP

Features / artifacts	TCP [Transmission Control Protocol]	UDP [User Datagram Protocol]
Connection	connection-oriented protocol.	connectionless protocol.
Usages	suited for applications that require high reliability, and transmission time is relatively less critical. Example - HTTP, HTTPS, FTP, SMTP, Telnet	Suitable for applications that need fast, efficient transmission, such as games. UDP's stateless nature is also useful for servers that answer small queries from huge numbers of clients. Example - DNS, DHCP, TFTP, SNMP, RIP, VOIP
Packet Ordering	rearranges data packets in specified order.	all packets are independent of each other. If ordering is required, it has to be managed by the application layer
Transfer Speed	slower than UDP.	UDP is faster / no error-checking for packets.
Reliability	There is absolute guarantee that the data transferred remains intact and arrives in the same order in which it was sent.	There is no guarantee that the messages or packets sent would reach at all.

TCP/IP and UDP

Features / artifacts	TCP [Transmission Control Protocol]	UDP [User Datagram Protocol]
Header	TCP header size is 20 bytes	UDP Header size is 8 bytes.
Common Header Fields	Source port, Destination port, Check Sum	Source port, Destination port, Check Sum
Streaming of data	Data is read as a byte stream, no distinguishing indications are transmitted to signal message (segment) boundaries.	Packets are sent individually and are checked for integrity only if they arrive. Packets have definite boundaries which are honored upon receipt, meaning a read operation at the receiver socket will yield an entire message as it was originally sent.
Weight	TCP is heavy-weight. TCP requires three packets to set up a socket connection, before any user data can be sent. TCP handles reliability and congestion control.	UDP is lightweight. There is no ordering of messages, no tracking connections, etc. It is a small transport layer designed on top of IP.

TCP and UDP Overview

Features / artifacts	TCP [Transmission Control Protocol]	UDP [User Datagram Protocol]
Data Flow Control	TCP does Flow Control. TCP requires three packets to set up a socket connection, before any user data can be sent. TCP handles reliability and congestion control.	UDP does not have an option for flow control
Error Checking	TCP does error checking	UDP does error checking, but no recovery options.
Fields	1. Sequence Number, 2. Ack number, 3. Data offset, 4. Reserved, 5. Control bit, 6. Window, 7. Urgent Pointer 8. Options, 9. Padding, 10. Check Sum, 11. Source port, 12. Destination port	1. Length, 2. Source port, 3. Destination port, 4. Check Sum
Acknowledgement	Acknowledgement segments	No Acknowledgment

Socket Overview

- ✓ A socket is a software endpoint that establishes bidirectional communication between a server program and one or more client programs.
- ✓ The combination of an IP address and a port number.
- ✓ Sockets provide an interface for programming networks at the transport layer.
- ✓ Two types
 - **Stream socket : reliable two-way connected communication streams**
 - **Datagram socket**

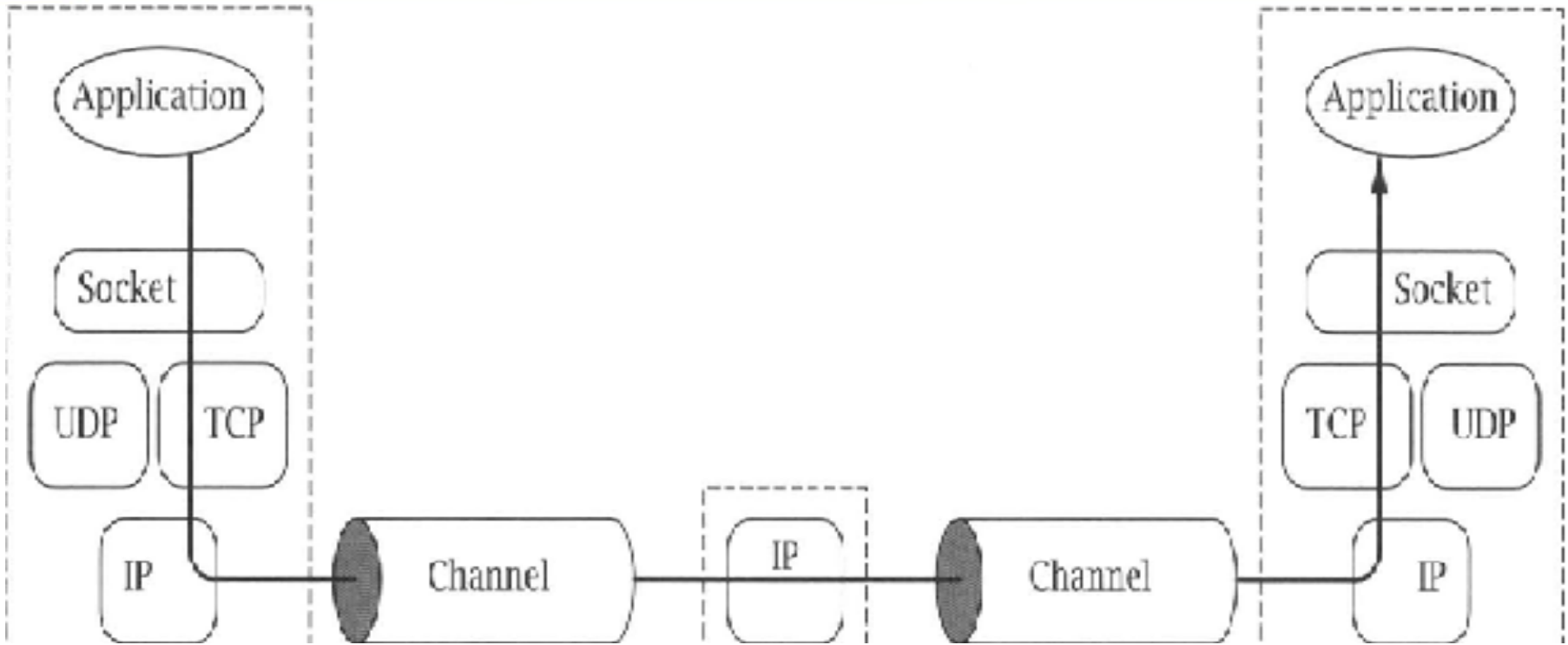
Socket pair

- ✓ Specified the two end points that uniquely identifies each TCP connection in an internet.
- ✓ 4-tuple: (client IP address, client port number, server IP address, server port number)

Socket Function call

- ✓ `socket ()`: Create a socket
- ✓ `bind()`: bind a socket to a local IP address and port #
- ✓ `connect()`: initiating connection to another socket
- ✓ `accept()`: accept a new connection
- ✓ `Write()`: write data to a socket
- ✓ `Read()`: read data from a socket
- ✓ `sendto()`: send a datagram to another UDP socket
- ✓ `recvfrom()`: read a datagram from a UDP socket
- ✓ `close()`: close a socket (tear down the connection)

TCP /IP Network



Java socket Programming Promises

java.net Package Overview

The Classes

- ✓ ContentHandler
- ✓ DatagramPacket
- ✓ DatagramSocket
- ✓ DatagramSocketImpl
- ✓ HttpURLConnection
- ✓ InetAddress
- ✓ MulticastSocket
- ✓ ServerSocket
- ✓ Socket
- ✓ SocketImpl
- ✓ URLConnection
- ✓ URL
- ✓ URLEncoder
- ✓ URLStreamHandler

The Interfaces

- ✓ ContentHandlerFactory
- ✓ FileNameMap
- ✓ SocketImplFactory
- ✓ URLStreamHandlerFactory

Exceptions

- ✓ BindException
- ✓ ConnectException
- ✓ MalformedURLException
- ✓ NoRouteToHostException
- ✓ ProtocolException
- ✓ SocketException
- ✓ UnknownHostException
- ✓ UnknownServiceException

Java Sockets Programming

- ✓ The package `java.net` provides support for sockets programming.
- ✓ Typically you import everything defined in this package with: **`import java.net.*;`**

✓ Important Classes

- `InetAddress`
- `Socket`
- `ServerSocket`
- `DatagramSocket`
- `DatagramPacket`
- `URL`

InetAddress Class

- ✓ InetAddress class is used to encapsulate IP address and host name.
- ✓ InetAddress class represents an Internet Protocol (IP) address.
- ✓ InetAddress has Factory Methods to access IP and Host Name.
- ✓ Throws `UnknownHostException`

Methods	Description
<code>static InetAddress getByAddress(byte[] addr)</code>	InetAddress object given the raw IP address
<code>static InetAddress getByName(String host)</code>	IP address of a host
<code>String getHostAddress()</code>	IP address string in textual presentation
<code>String getLocalHost()</code>	local host
<code>static InetAddress getLocalHost()</code>	local host

InetAddress class Example

```
import java.net.InetAddress;
import java.net.UnknownHostException;

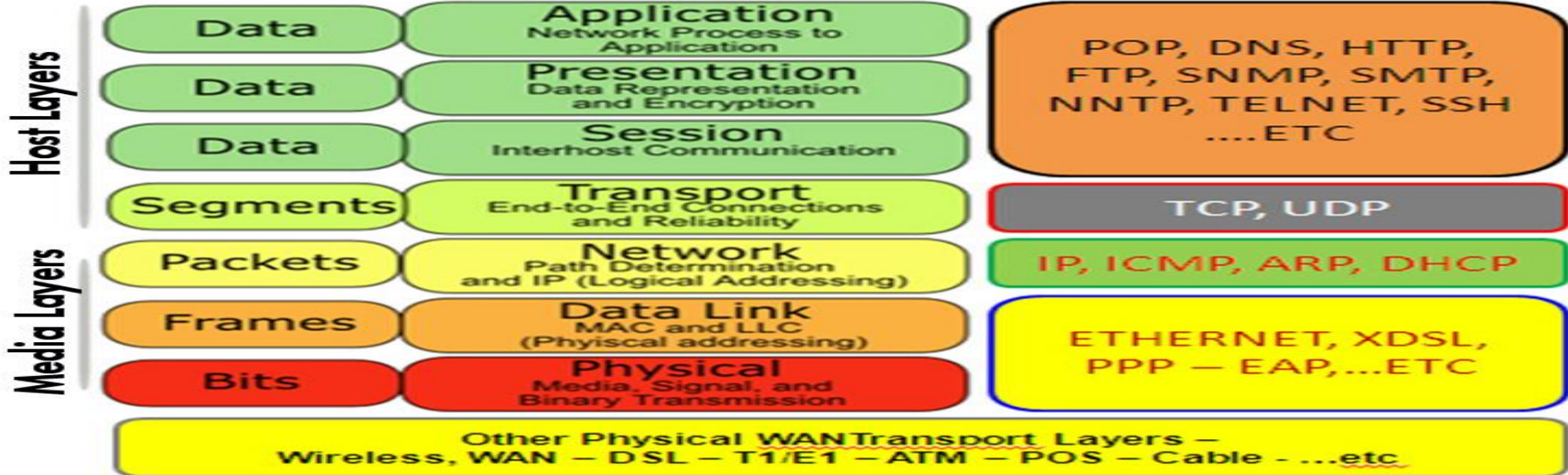
public class InetAddressTest {
    public static void main(String[] args) throws UnknownHostException {
        InetAddress address;
        InetAddress localhost= InetAddress.getLocalHost();
        System.out.println(localhost.getHostName() + ":" + localhost.getHostAddress());
        System.out.println(localhost.getCanonicalHostName()); // fully qualified Name
        address= InetAddress.getByName("www.cdac.in");

        System.out.println(address +": " +address.getHostAddress());
        InetAddress addr[]= InetAddress.getAllByName("www.google.com");
        for (InetAddress addr1 : addr) {
            System.out.println(addr1);
        }
    }
}
```

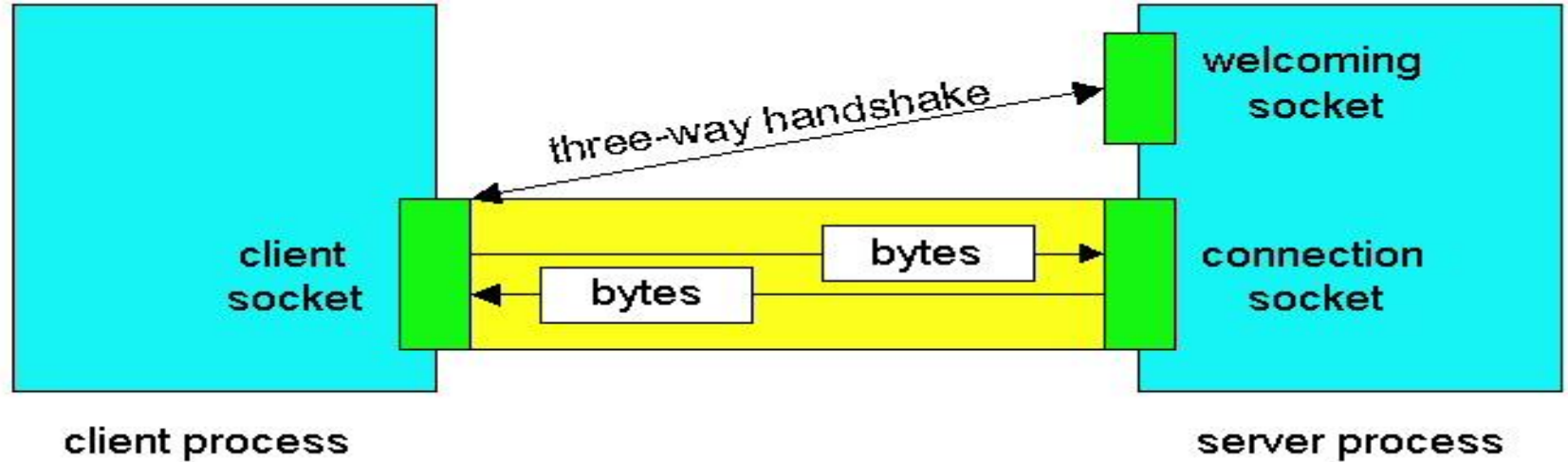
Socket Programming –TCP/IP

TCP/IP Stack

OSI Example for Ethernet Media - TCP/IP STACK



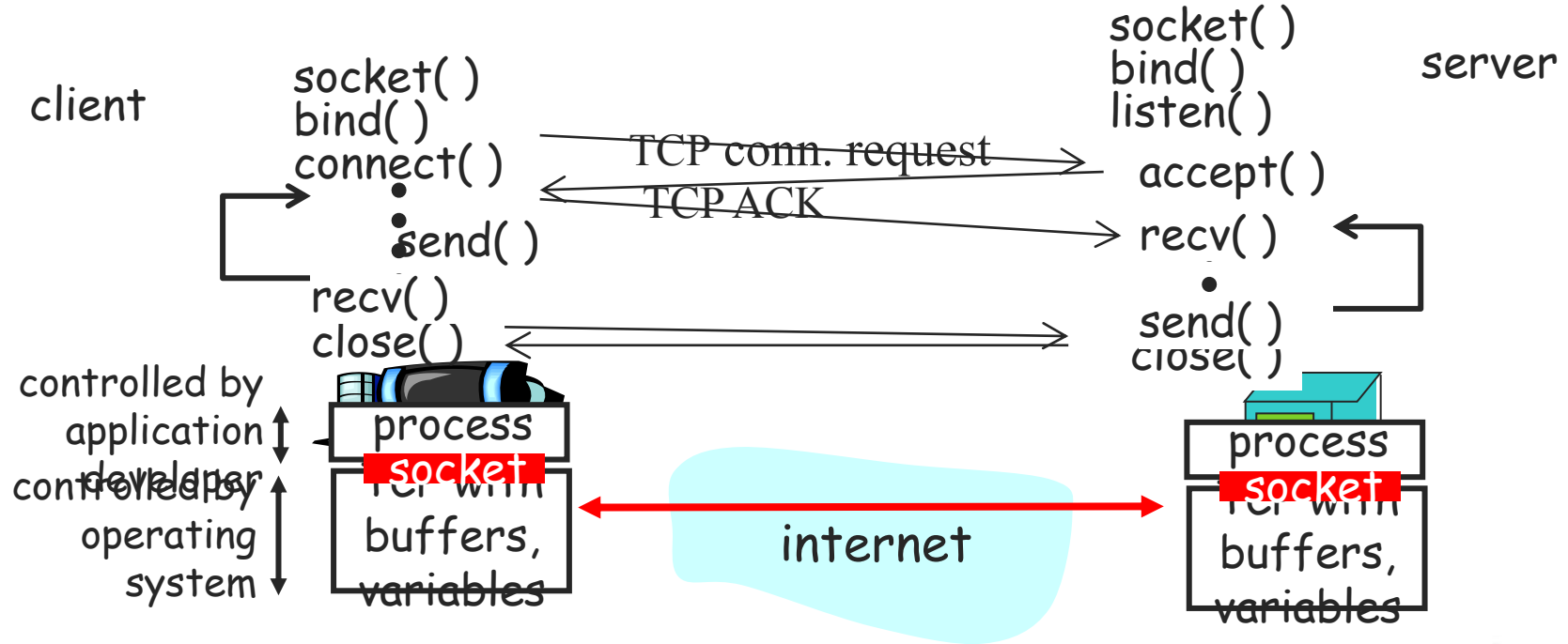
Sockets with TCP/IP



Client socket, welcoming socket (passive) and connection socket (active)

Socket-programming using TCP

TCP service: reliable byte stream transfer



JAVA TCP Sockets Programming

✓ java.net.Socket class :

- Implements client sockets (also called just “sockets”).
- An endpoint for communication between two machines.
- Constructor and Methods

- ❖ `Socket(String host, int port)`: Creates a stream socket and connects it to the specified port number on the named host.
- ❖ `InputStream getInputStream()`
- ❖ `OutputStream getOutputStream()`
- ❖ `close()`

✓ java.net.ServerSocket class :

- Implements server sockets.
- Waits for requests to come in over the network.
- Performs some operation based on the request.
- Constructor and Methods

Sockets class

- ✓ The **java.net.Socket** class represents the socket that both the client and server **use to** communicate with each other.
- ✓ The client obtains a Socket object by instantiating one, whereas the server obtains a Socket object from the return value of the `accept()` method.

Sockets class

	Constructors and Methods
1.	<code>Socket(InetAddress server, int port)</code>
2.	<code>Socket(InetAddress server, int port, Address local, int localport);</code>
3.	<code>Socket(String hostname, int port);</code>
4.	<code>InetAddress getInetAddress ()</code>
5.	<code>Int getport ()</code>
6.	<code>Int getLocalPort()</code>
7.	<code>InputStream getInputStream();</code>
8.	<code>OutputStream getOutputStream();</code>
9.	<code>void close();</code>

Server Socket

java.net.ServerSocket is used to implement Server

Creating a ServerSocket

```
ServerSocket serverSocket = new ServerSocket(9000);
```

Listening For Incoming Connections

```
Socket clientSocket = serverSocket.accept(); // normal socket
```

ServerSocket class

	Constructors
1.	ServerSocket(int port) throws IOException
2.	Socket(InetAddress server, int port, Address local, int localport);
3.	ServerSocket(int port, int backlog) throws IOException
4.	ServerSocket(int port, int backlog) throws IOException
5.	ServerSocket(int port, int backlog, InetAddress address) throws IOException
6.	Int getLocalPort()
7.	public ServerSocket() throws IOException

ServerSocket

	Methods
1.	public int getLocalPort()
2.	Socket accept() throws IOException
3.	public void bind(SocketAddress host, int backlog)
4.	public void setSoTimeout(int timeout)
5.	ServerSocket(int port, int backlog, InetAddress address) throws IOException
6.	int getLocalPort()
7.	public ServerSocket() throws IOException

Client/server Socket interaction: TCP

Server (running on **host id**)

Client

create socket,
port=**x**, for
incoming request:
welcomeSocket =
ServerSocket()

wait for incoming
connection request
connectionSocket =
welcomeSocket.accept()

read request from
connectionSocket

write reply to
connectionSocket

close
connectionSocket

TCP
connection setup

create socket,
connect to **hostid**, port=**x**
clientSocket =
Socket()

read reply from
clientSocket
close
clientSocket

TCP Server Example

```
public class TCPServer {  
  
    public static void main(String[] args) {  
        ServerSocket welcomeSock = null;  
        try {  
            welcomeSock = new ServerSocket(5555);  
            echo("Server is waiting for incoming request ....");  
            while (true) {  
                Socket socket = welcomeSock.accept();  
                // Read  
                DataInputStream dis = new DataInputStream(socket.getInputStream());  
                String s = dis.readUTF();  
                echo(socket.getInetAddress().getHostName() + ":" + socket.getPort() + "--- " + s);  
                s = s.toUpperCase();  
                // write  
                DataOutputStream dos = new DataOutputStream(socket.getOutputStream());  
                dos.writeUTF(s);  
            }  
        } catch (IOException e) { }  
    }  
}
```

TCP Client Example

```
public class TCPClient {  
  
    public static void main(String[] args) {  
        Socket sock = null;  
        DataOutputStream dos = null;  
        DataInputStream dis = null;  
  
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
        echo("Enter sentence : ");  
        while (true) {  
            try {  
                String s = br.readLine();  
                sock = new Socket("localhost", 5555);  
                dos = new DataOutputStream(sock.getOutputStream());  
                dos.writeUTF(s);  
                dis = new DataInputStream(sock.getInputStream());  
                s = dis.readUTF();  
                echo("modified string : " + s);  
  
            } catch (IOException e) { }  
        }  
    }  
}
```

Network Programming –UDP

UDP Sockets

UDP

- ✓ Connectionless and unreliable service.
- ✓ There isn't an initial handshaking phase.
- ✓ Doesn't have a pipe or stream.
- ✓ transmitted data may be received out of order, or lost

Socket Programming with UDP

- ✓ No need for a welcoming socket.
- ✓ No streams are attached to the sockets.
- ✓ the sending hosts creates “packets” by attaching the IP destination address and port number to each batch of bytes.
- ✓ The receiving process must unravel to received packet to obtain the packet's information bytes.

UDP Sockets

- ✓ A datagram socket is the sending or receiving point for a packet delivery service.
- ✓ Each packet sent or received on a datagram socket is individually addressed and routed.
- ✓ Multiple packets sent from one machine to another may be routed differently, and may arrive in any order.
- ✓ **DatagramPacket** class needed to specify the payload incoming or outgoing

DatagramSocket Class

Socket Programming using UDP is realized using

✓ **DatagramSocket** class

public class **DatagramSocket** extends Object implements Closeable

Creating Datagram Socket

```
DatagramSocket s = new DatagramSocket(null);  
s.bind(new InetSocketAddress(8888)) ;
```

```
DatagramSocket s = new DatagramSocket(8888)
```



Datagram Socket Constructor

Constructors

```
DatagramSocket() ;
```

Constructs a datagram socket and binds it to any available port on the local host machine

```
DatagramSocket(int port, InetAddress a) ;
```

Creates a datagram socket, bound to the specified local address.

```
DatagramSocket(int port) ;
```

Constructs a datagram socket and binds it to the specified port on the local host machine.

All can throw `SocketException` or `SecurityException`

Datagram Socket Methods

Methods

public void bind(SocketAddress addr) throws [SocketException](#)

Binds this DatagramSocket to a specific address & port.

void connect(InetAddress, int port);

Connects the socket to a remote address for this socket. When a socket is connected to a remote address, packets may only be sent to or received from that address. By default a datagram socket is not connected.

void close();

void receive(DatagramPacket p);

Receives a datagram packet from this socket

void send(DatagramPacket p);

Sends a datagram packet from this socket

DatagramPacket

- ✓ A datagram is an independent, self-contained message sent over the network whose arrival, arrival time, and content are not guaranteed.
- ✓ Datagram packets are used to implement a connectionless packet delivery service. Each message is routed from one machine to another based **solely on information contained within that packet**.
- ✓ Multiple packets sent from one machine to another might be routed differently, and might arrive in any order. Packet delivery is not guaranteed.
- ✓ Contain the payload (a byte array can also be used to specify the destination address when not using connected mode UDP).

public final class **DatagramPacket** extends [Object](#)

DatagramPacket Constructors

DatagramPacket(byte[] buf, int length)

Constructs a DatagramPacket for receiving packets of length length.

DatagramPacket(byte[] buf, int length, **InetAddress** address, int port)

Constructs a datagram packet for sending packets of length to the specified port number on the specified host.

DatagramPacket Important Methods

public byte[] getData()

Returns the data buffer. The data received or the data to be sent starts from the offset in the buffer, and runs for length long.

public void setData(byte[] buf)

Set the data buffer for this packet. With the offset of this DatagramPacket set to 0, and the length set to the length of buf.

void setAddress(InetAddress a);

Sets the IP address of the machine to which this datagram is being sent.

void setPort(int port);

Sets the port number on the remote host to which this datagram is being sent.

InetAddress getAddress();

Returns IP address of the machine to which this datagram is being sent or from which the datagram was received.

int getPort();

port number on the remote host to which this datagram is being sent or from which the datagram was received.

DatagramPacket

For receiving:

```
DatagramPacket( byte[] buf, int len);
```

For sending:

```
DatagramPacket( byte[] buf, int len  
                InetAddress a, int port);
```


UDP Server and Client communication Steps

Create UDP Socket bind it with port number

// To Receive

Create Datagram packet for receiving data

Perform receive operation from socket

// To send

Create Datagram packet and pass InetAddress and port

Perform send operation from socket

Client/server socket interaction: UDP

Server (running on **hostid**)

create socket,
port=**x**, for
incoming request:
serverSocket =
DatagramSocket()

read request from
serverSocket
write reply to
serverSocket

specifying client
host address,
port number

Client

create socket,
clientSocket =
DatagramSocket()

Create, address (**hostid**, **port=x**,
send datagram request
using **clientSocket**

read reply from
clientSocket
close
clientSocket

UDP Server Example

```
public class UDPServer {
public static void main(String[] args) throws IOException {
byte[] sendData = new byte[1024];
byte[] recieveData = new byte[1024];

try {
DatagramSocket socket = new DatagramSocket(3333);

while (true) {
// Recieve Data
DatagramPacket recievedPckt = new DatagramPacket(recieveData, recieveData.length);
socket.receive(recievedPckt);
String sentence = new String(recievedPckt.getData());
System.out.println("Recieved from Client :" + sentence);
// To send
InetAddress addr = recievedPckt.getAddress();
int port = recievedPckt.getPort();
String captialsentence = sentence.toUpperCase();
sendData = captialsentence.getBytes();
DatagramPacket sendpckt = new DatagramPacket(sendData, sendData.length, addr, port);
socket.send(sendpckt);
}
} catch (SocketException ex) { }
}
```

UDP Client Example

```
public class UDPClient {  
  
    public static void main(String[] args) throws SocketException, IOException {  
        byte[] sendData = new byte[1024];  
        byte[] recieveData = new byte[1024];  
  
        // Take input from user  
        DatagramSocket clientSocket = new DatagramSocket();  
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
  
        String sentence = br.readLine();  
        // Send data to server  
        sendData = sentence.getBytes();  
        InetAddress addr = InetAddress.getByName("localhost");  
        DatagramPacket sendpacket = new DatagramPacket(sendData, sendData.length, addr, 3333);  
        clientSocket.send(sendpacket);  
  
        // recieve Data  
  
        DatagramPacket recvpacket = new DatagramPacket(recieveData, recieveData.length);  
        clientSocket.receive(recvpacket);  
        String modifiedStr = new String(recvpacket.getData());  
        System.out.println("modified :" + modifiedStr);  
    }  
}
```

URL, URLConnection

URL

URL class represents a Uniform Resource Locator, a pointer to a "resource" on the World Wide Web.

A resource can be something as simple as a file or a directory, or it can be a reference to a more complicated object, such as a query to a database or to a search engine.

URL class Constructors

	Constructors
1	URL (String UriSpeccifier) throws MalformedURLException
2	URL (String protocol,String hostName, int port, String path) throws MalformedURLException
3	URL (String protocolName, String hostName, String path) throws MalformedURLException

URL class Important methods

	Methods	Description
1	<code>public String getProtocol()</code>	it returns the protocol of the URL.
2	<code>public String getHost()</code>	it returns the host name of the URL.
3	<code>public String getPort()</code>	it returns the Port Number of the URL.
4.	<code>public String getFile()</code>	it returns the file name of the URL
5.	<code>public URLConnection openConnection()</code>	It returns URLConnection
5.	<code>public Object getContent()</code>	It return content of the URL

URL Example

```
public class URLLDemo {  
  
    public static void main(String[] args) throws MalformedURLException {  
  
        URL url = new URL("http://www.cdac.in");  
  
        System.out.println("Protocol: " + url.getProtocol());  
        System.out.println("Host Name: " + url.getHost());  
        System.out.println("Port Number: " + url.getPort());  
        System.out.println("File Name: " + url.getFile());  
        System.out.println("EXt: " + url.toExternalForm());  
  
    }  
}
```

URLConnection Class

- ✓ The abstract class `URLConnection` is the super class of all classes that represent a communications link between the application and a URL.
- ✓ Instances of this class can be used both to read from and to write to the resource referenced by the URL.
- ✓ In general, creating a connection to a URL is a multistep process:
 - The connection object is created by invoking the `openConnection` method on a URL.
 - The setup parameters and general request properties are manipulated.
 - The actual connection to the remote object is made, using the `connect` method.
 - The remote object becomes available. The header fields and the contents of the remote object can be accessed.

URLConnection Class

- ✓ The following methods are used to access the header fields and the contents after the connection is made to the remote object:
 - getContent
 - getHeaderField
 - getInputStream
 - getOutputStream

- ✓ Certain header fields are accessed frequently. The following methods provide convenient access
 - getContentEncoding
 - getContentLength
 - getContentType
 - getDate
 - getExpiration
 - getLastModified

URLConnection Class

```
public class URLConnectionDemo {  
  
    public static void main(String[] args) {  
  
        try {  
            URL url = new URL("http://www.cdac.in");  
            URLConnection urlcon = url.openConnection();  
            InputStream stream = urlcon.getInputStream();  
            int i;  
            while ((i = stream.read()) != -1) {  
                System.out.print((char) i);  
            }  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

Assignments

1. Devolve a Multi threaded **chat server** using TCP/UDP socket communication.
1. Develop a client and server program using TCP socket communication and Server should offer following services. Whenever client sends a number to client sever should test
 - a. Odd/even Test
 - b. Prime Number Test
2. Develop a client and server program using UDP socket Communication and server should offer following service. Whenever client sends a String and server should return **capitalized** String.
3. Develop a client and server program using UDP socket Communication and server should offer following service. Whenever client sends a number it should return **Fibonacci series**.

Assignments

1. Devolve a Multi threaded **chat server** using TCP/UDP socket communication.
1. Develop a client and server program using TCP socket communication and Server should offer following services. Whenever client sends a number to client sever should test
 - a. Odd/even Test
 - b. Prime Number Test
2. Develop a client and server program using UDP socket Communication and server should offer following service. Whenever client sends a String and server should return **capitalized** String.
3. Develop a client and server program using UDP socket Communication and server should offer following service. Whenever client sends a number it should return **Fibonacci series**.

Further Reading

- ✓ <http://docs.oracle.com/javase/tutorial/networking/>

